20. Package Management

Created by suncaper, last modified about 3 hours ago

Introduction

In the lesson on Modules & Imports, we learned how to use the `import` keyword to bring pre-written code (like `math` or `datetime`) into our scripts. But those modules are part of Python's "Standard Library"—they come pre-installed when you download Python.

What happens when you want to use **Apache Spark** (`pyspark`), connect to AWS (`boto3`), or pull data from an API using the famous `requests` library? These do not come with Python. You have to download them.

PIP (Package Installer for Python) is the command-line tool used to download and install external libraries from the internet.

In Data Engineering, managing your "dependencies" (the external packages your code depends on to run) is absolutely critical. If you write a script on your laptop that uses `pandas`, but you forget to install `pandas` on your production server, your pipeline will crash instantly.

In this lesson, we will cover:

- What PyPI is (The Python App Store).
- Basic terminal commands (`pip install`, `pip list`).
- The almighty `requirements.txt` file.
- Why dependency management is the secret to stable Big Data clusters.

Part 1: PyPI and The Command Line

PyPI (Python Package Index) is essentially the App Store for Python. It is a massive, public database of over 400,000 open-source projects.

PIP is the tool that connects to PyPI, downloads the package you request, and installs it onto your machine.

Crucial Note: You do not run PIP commands inside your Python script (e.g., you don't type it in PyCharm's Python editor). You run PIP in your **Terminal** or **Command Prompt**.

Part 2: Essential PIP Commands

1. Installing a Package

To install a library, open your terminal and type:

```
pip install requests
```

Python reaches out to PyPI, downloads the `requests` library, and makes it available for you to `import` into any of your Python scripts.

2. Installing a Specific Version

In data pipelines, upgrading to a brand new version of a package can sometimes break your code if the creators changed how the functions work. Data Engineers usually lock in specific versions.

```
pip install pandas==2.0.3
```

3. Seeing What is Installed

To see every external package currently installed on your system (and their versions), use:

```
pip freeze
```

4. Example-requests

```
import requests

# 1. Target Open API URL (using a free test API to fetch a list of users)
url = "https://jsonplaceholder.typicode.com/users"

print("Sending request...")

# 2. Send the most basic GET request
response = requests.get(url)

# 3. Check if the HTTP status code is 200 (200 indicates a successful request)
if response.status_code == 200:
    # 4. Parse the returned content directly into a Python dictionary or list
    data = response.json()

    print("✅ Success! Here is the data for the first 3 users:\n")

    # 5. Iterate through and extract the data we need
    for user in data[:3]:
        print(f"Name: {user['name']}")
        print(f"Email: {user['email']}")
        print("-" * 20)
else:
    print(f"❌ Request failed, status code: {response.status_code}")
```

Part 3: The "Big Data" Context (Why do we care?)

1. **The `requirements.txt` File:** When you deploy your ETL code to a cloud server, the server is a blank slate. How does it know which packages to download? Data Engineers generate a file named `requirements.txt` containing a list of every package their project needs. When the server boots up, it runs `pip install -r requirements.txt` to automatically install everything before the pipeline starts.
2. **Cluster Initialization (Spark/Databricks):** If you are running a Spark job across 50 worker nodes, every single node must have the exact same packages installed. Cloud platforms (like AWS EMR or Databricks) require you to submit your PIP dependencies so they can distribute the installations across the cluster during boot-up.
3. **Airflow Providers:** If you use Apache Airflow to schedule your jobs, it only comes with basic features. If you want Airflow to connect to Snowflake, you must run `pip install apache-airflow-providers-snowflake` on your Airflow server.

Part 4: Hands-on Exercise (Simulating Dependency Management)

Let's step outside of pure Python for a moment and simulate setting up a new project environment. We will use the terminal to install a third-party package (`requests`), create a requirements file, and then write a tiny Python script to prove the package works.

Step A: The Terminal (Package Installation)

1. Open PyCharm (or your terminal/command prompt).
2. Open the **Terminal** tab at the bottom of the screen.
3. Type the following commands and hit Enter after each:

```
# 1. Install the 'requests' library (used for extracting API data)
pip install requests

# 2. Save your current environment's packages to a text file
pip freeze > requirements.txt
```

If you look at your project files, you will now see a `requirements.txt` file! This is what you would hand over to your DevOps team.

Step B: The Python Script (Extracting Data)

1. Create a new Python file named `pip_practice.py`.
2. Type the following code to utilize the package you just installed:

```
# ---------------------------------------------------------
# Big Data Simulation: Third-Party API Extraction
# ---------------------------------------------------------

"""
Scenario: We need to extract data from a public REST API.
Python's built-in tools for this are clunky. We will use the
third-party 'requests' package we just installed via PIP.
"""

# 1. IMPORT: Bring in the external package
import requests

print("--- Starting API Extraction ---")

try:
    # 2. EXTRACT: Call a public test API to get a fake user record
    # (This function is provided by the 'requests' library)
    response = requests.get("https://jsonplaceholder.typicode.com/users/1")

    # Check if the API call was successful (Status Code 200)
    if response.status_code == 200:
        print("SUCCESS: Connected to API.")

        # 3. TRANSFORM: Convert the raw JSON response into a Python Dictionary!
        user_data = response.json()

        # Extract specific data using dictionary syntax
        extracted_name = user_data.get("name")
        extracted_city = user_data["address"]["city"]

        # 4. LOAD / OUTPUT
        print("Extracted User: " + extracted_name)
```

```
        print("Location: " + extracted_city)
    else:
        print("FAILED: API returned status code " + str(response.status_code))

except Exception as e:
    print("CRITICAL ERROR: Is the 'requests' library installed?")
    print(e)


Run the code. Output:

Plaintext

--- Starting API Extraction ---
SUCCESS: Connected to API.
Extracted User: Leanne Graham
Location: Gwenborough
```

What happens?

- Because you ran `pip install requests` in your terminal, Python is able to successfully execute `import requests` at the top of the script.
- The script uses the external library's `.get()` tool to ping a live URL over the internet.
- It receives a JSON payload, converts it into a familiar Python Dictionary, and uses nested dictionary keys (from Lesson 09) to extract the target data.

Summary

- **PyPI** is the public database of Python packages.
- **PIP** is the command-line tool used to install them (`pip install <package_name>`).
- **`requirements.txt`** is the standard file used to list every external package your pipeline needs to function.
- **Data Pipelines depend on PIP:** Your data engineering code will rely heavily on external libraries like `pandas`, `boto3`, `sqlalchemy`, and `pyspark`. Mastering PIP ensures your code runs flawlessly across different servers and cloud environments.

No labels